

Project File

INT232

Course: Data Science ToolBox: R Programming



School of Chemical Engineering & Physical Sciences

Department OF Statistics and Data Analyst

Title: Myntra Smart Dashboard

SUBMITTED BY: KRITIKA THAKUR

REGISTRATION NO- 12529249

SUBMITTED TO: PARUL PRAKRAM SHARMA

SECTION: G3504

1. Objective

This project aims to build a responsive online interface through R Shiny, designed for navigating the Myntra product dataset interactively. With each adjustment - be it category, brand, cost bracket, or required star score - the display shifts fluidly. Response times remain immediate; performance does not lag under selection changes. Key indicators shift alongside visual summaries when inputs are modified. A suggestions module appears below, shaped by current filters applied above. Functionality centers on live exploration, avoiding static displays entirely. User choices drive every update without delays between steps. Visuals adapt silently, guided only by selected constraints. Behind the layout, logic processes selections efficiently. Interaction forms the core mechanism, replacing manual queries completely. Output evolves moment by moment as new conditions emerge. Design avoids clutter, focusing strictly on active data paths. Navigation feels continuous, never fragmented into separate views. Elements align dynamically, responding solely to parameter adjustments. Insights arise directly from filtered subsets at any given instant.

2. Introduction

Hundreds of thousands of products sit on e-commerce sites such as Myntra, spread through many categories. Because the volume is so high, manual review becomes unworkable. Visual exploration turns complexity into clarity - dashboards make that possible. With them, patterns emerge quickly, without sifting through rows of numbers. Insights appear not after long effort, but nearly at once.

Open-source tools like R Shiny let users create live web apps using only R code. Not knowing HTML, CSS, or JavaScript becomes irrelevant when constructing these interfaces. Libraries including Plotly bring moving visuals into view; DT adds tables that respond to input. For school-related projects or workplace reports, such setups serve well. Functionality grows once pieces connect beneath one interface.

This project relies on a real-world collection of 168,034 Myntra items, displayed using four performance indicators alongside interactive visualizations. Rather than simple summaries, it incorporates dynamic charts tied to filtering tools that adapt with user input. Insights appear contextually, avoiding static placement, while recommendations emerge from pattern recognition within the set. Export functionality allows external review, maintaining data integrity throughout transfers.

3. Tools & Technologies Used

Tool / Library	Purpose
R Programming	Base language for all logic and computation
RStudio	Integrated development environment (IDE)
Shiny	Web application framework — UI + Server
shinydashboard	Dashboard layout (header, sidebar, body)

tidyverse (dplyr)	Data cleaning, filtering, and transformation
ggplot2	Static chart generation (inside Plotly wrapper)
Plotly	Interactive, hover-enabled chart rendering
DT	Interactive, searchable, paginated data tables

4. Dataset Description

The dataset used in this project is myntra.csv, a real-world product catalog downloaded from Myntra, one of India's largest fashion e-commerce platforms. It contains 168,034 rows and 13 columns.

Column Name	Data Type	Description
product_name	Character	Full name of the product listing
brand_name	Character	Brand that makes the product (e.g. Roadster, H&M)
rating	Numeric	Average customer rating (1.0 to 5.0)
rating_count	Numeric	Number of customer ratings received
marked_price	Character → Numeric	Original listed price before discount
discounted_price	Character → Numeric	Final price after applying the discount
discount_percent	Character → Numeric	Percentage discount offered (e.g. 36%)
discount_amount	Numeric	Amount saved in rupees
sizes	Character	Available sizes (S, M, L, XL, etc.)
product_tag	Character	Product category (e.g. tshirts, dresses, kurtas)
brand_tag	Character	URL-safe brand slug
product_link	Character	URL to the product page on Myntra
img_link	Character	URL to the product image

Key Dataset Statistics

Total Products: 168,034

Unique Brands: 3,194

Avg Price: ₹1,515

Avg Discount: 37.3%

Avg Rating: 4.17 out of 5.0

Top Category: T-Shirts (16,978 products)

Top Brand: Roadster (4,982 products)

5. Methodology

The project was developed in the following steps:

- From a single source emerged the Myntra product collection - 168,034 entries stored within myntra.csv. Each record arrived pre-labeled, forming a base layer for structured review.
- Unrated items were excluded first - those showing a zero rating. Text formatting interfered with analysis, so symbols like commas and percent signs were extracted from pricing fields through string substitution. Conversion followed, transforming cleaned character entries into numerical format for computation. Missing data posed issues in essential variables; thus, records containing nulls were omitted. Processing concluded once inconsistent or incomplete cases no longer remained.
- At initialization, fixed worldwide pricing limits are established. These values, named PRICE_MIN and PRICE_MAX, support range adjustments later. The slider uses them directly. Reset operations rely on these preset extremes. Their presence ensures consistent behaviour across interactions.
- A dark-themed dashboardPage formed the base of the Shiny UI layout. Within it, a sidebar appeared containing four distinct filters: category, brand, price range, alongside minimum rating. A reset option followed toward the lower edge of the panel. Toward the right, a download function was positioned for user access. Visual balance emerged through deliberate spacing around interactive elements.
- Server-side logic now runs reactively. Whenever inputs change, filtered_data() activates. This function combines four criteria at once using logical checks instead of sequential steps. In cases where selections exclude every item, the system defaults to showing all available entries within the chosen group. Results update without reloading the page. Logic flow remains consistent even during edge cases like empty states. Processing happens before display, ensuring only relevant records appear. Full dataset access restores automatically under zero-match conditions.
- A change in category now triggers an event. This adjustment filters the available brands within that group. The selection range for prices adapts accordingly. Only values relevant to the chosen type appear. Interface elements shift without requiring a page reload. Brand options update first. Then, pricing limits reset based on typical costs for that class. User focus remains uninterrupted during transitions.
- A total of four KPI value displays were constructed alongside interactive visualizations using Plotly. Following these, an area dedicated to generated insights was included. A tool suggesting top products appears further down the layout. Present also is a table of items that supports search queries. Found at the bottom lies functionality for exporting data as CSV files.
- Nine issues detected in testing have now been resolved. Crashes tied to NA values no longer occur. Slider behaviour adjusts correctly for whole numbers. The range validation logic has reversed its prior fault. Function guards using req() operate as intended. Raw 'NA' strings will not appear within KPI displays. Histogram rendering

handles uniform rating inputs without failure. Output modules avoid crashing when data is absent across all visual formats.

6. Dashboard Design

The dashboard follows a three-layer architecture: UI Layer, Server Logic Layer, and Data Layer. It is built with shinydashboard's dashboardPage and uses a black skin for a modern dark theme.

Sidebar : Filter Panel

- Category Filter (selectInput) — sorted dropdown of all product categories
- Brand Filter (selectInput) — dynamically updates to show only brands in the selected category
- Price Range Filter (sliderInput) — rescales automatically per category; step = 1; displays ₹ symbol
- Minimum Rating Filter (sliderInput) — NEW: allows filtering to 4+ or 4.5+ rated products
- Reset Filters Button — restores all filters to their default values
- Download Button — exports current filtered dataset as a dated CSV file

Dashboard Body : Main Outputs

Section	Output	Data Source
KPI Row	Avg Rating, Avg Discount, Total Products, Avg Price	Filtered data
Status Bar	Warning when 0 results — shows fallback message	Reactive flag
Insights Panel	Auto-text: count, rating, discount, price range, best brand	Filtered data
Best Product	Top-rated + highest-discount product as a DT table	Filtered data
Chart 1	Top 10 Categories — horizontal bar chart	Full dataset
Chart 2	Price vs Discount — scatter plot with hover	Filtered data
Chart 3	Rating Distribution — dynamic histogram	Filtered data
Chart 4	Top Brands by Count — column chart with tooltips	Full dataset
Product Table	Searchable, paginated table sorted by discount %	Filtered data

7. Code Implementation

- Library Loading

```
library(shiny)
library(shinydashboard)
library(tidyverse)
library(DT)
library(plotly)
```

- Data Loading and Cleaning

```
#Load & Clean
df <- read.csv("D:/book_ch/R/myntra.csv")
df <- df %>%
  filter(rating > 0) %>%
  mutate(
    discount_percent = as.numeric(gsub("%", "", discount_percent)),
    discounted_price = as.numeric(gsub(",", "", discounted_price)),
    marked_price = as.numeric(gsub(",", "", marked_price))
  ) %>%
  # Remove rows with NA prices — these break slider range and filter()
  filter(!is.na(discounted_price), !is.na(marked_price), !is.na(discount_percent))
# Pre-compute global price bounds (used in slider and reset)
PRICE_MIN <- floor(min(df$discounted_price))
PRICE_MAX <- ceiling(max(df$discounted_price))
```

- Core Reactive Filter

```
filtered_data <- reactive({
  req(input$priceRange, input$category, input$brand, input$minRating)
  price_low <- input$priceRange[1]
  price_high <- input$priceRange[2]
  if (is.na(price_low) || is.na(price_high) || price_low > price_high) {
    price_low <- PRICE_MIN
    price_high <- PRICE_MAX
  }
  data_filtered <- df %>%
    filter(
      product_tag == input$category,
      discounted_price >= price_low,
      discounted_price <= price_high,
      rating >= input$minRating
    )
  if (input$brand != "All") {
    data_filtered <- data_filtered %>%
      filter(brand_name == input$brand)
  }
  # Fallback: if combined filters produce 0 rows, show full category
  if (nrow(data_filtered) == 0) {
    return(list(
      data = df %>% filter(product_tag == input$category),
      fallback = TRUE
    ))
  }
})
```

- Reactive Brand & Price Update

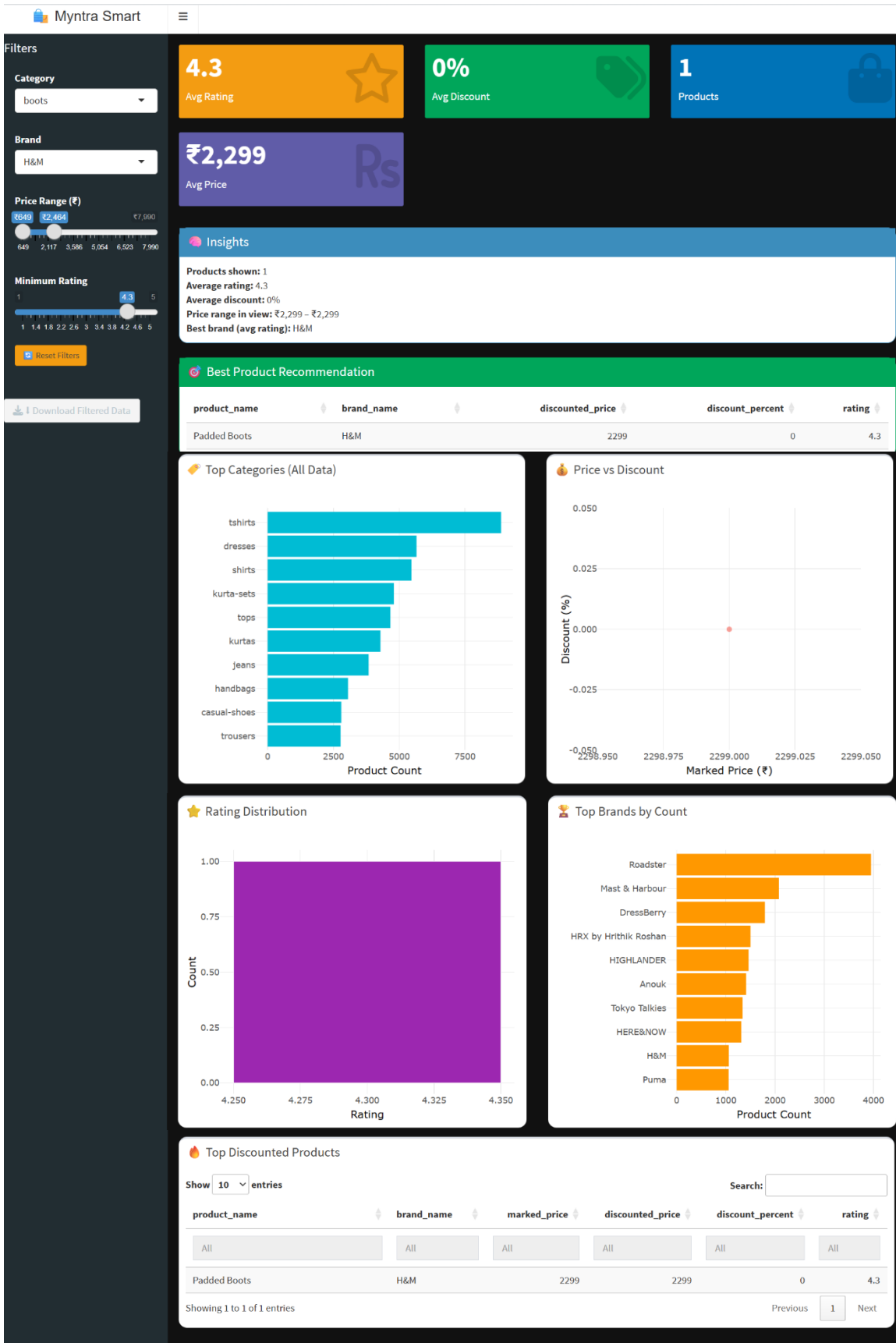
```
observeEvent(input$reset, {  
  first_cat <- sort(unique(df$product_tag))[1]  
  updateSelectInput(session, "category", selected = first_cat)  
  updateSelectInput(session, "brand", selected = "All")  
  # FIX: reset uses floor/ceiling consistently with slider definition  
  updateSliderInput(session, "priceRange",  
    min = PRICE_MIN,  
    max = PRICE_MAX,  
    value = c(PRICE_MIN, PRICE_MAX))  
  updateSliderInput(session, "minRating", value = 1)  
}
```

11

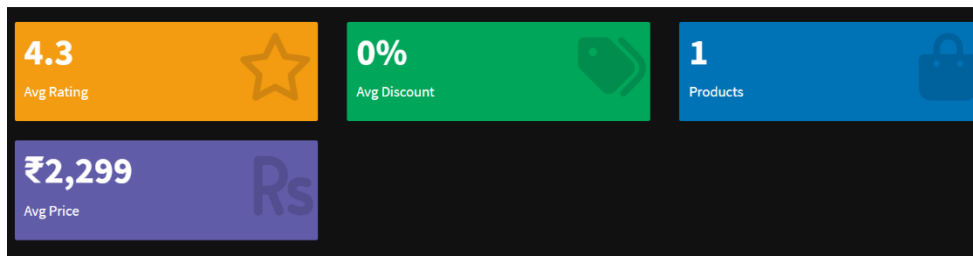
8. Output Screenshots

The following section contains screenshots of the running dashboard. Paste your actual screenshots below each description when you run the application in RStudio.

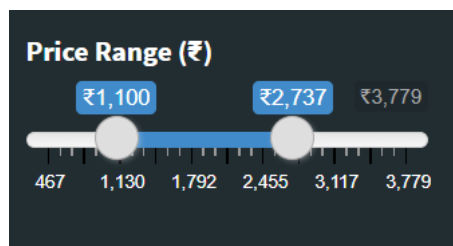
- Full Dashboard View



- KPI Value Boxes



- Price Range Filter in Action



- Category + Brand Filter

Filters

Category

concealer ▼

Brand

L'Oreal ▼

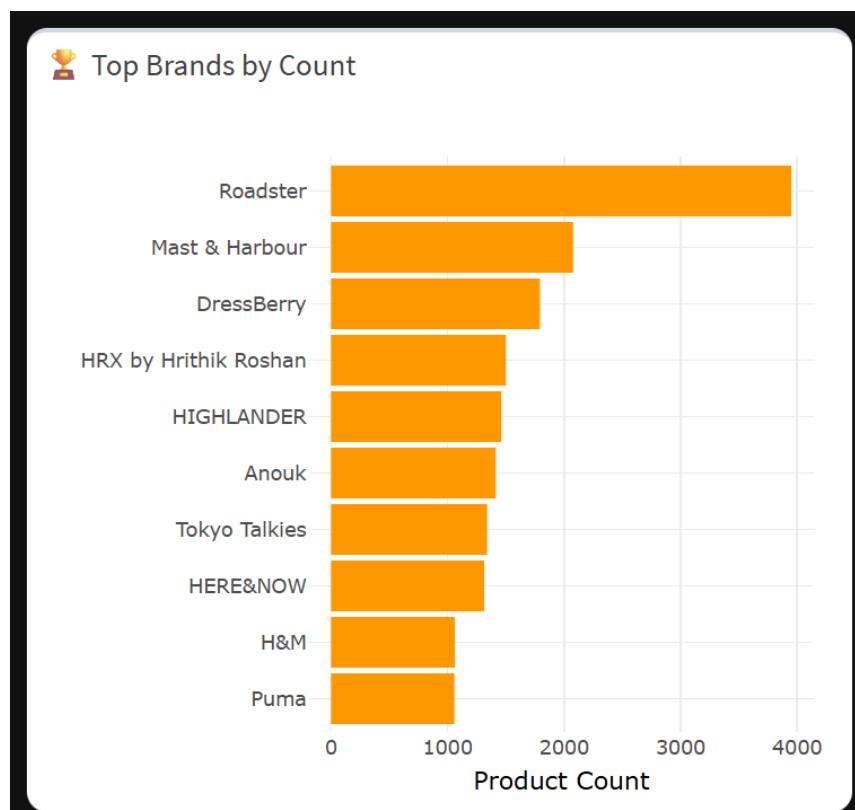
- Charts — Top Categories & Price vs Discount



- Rating Distribution Histogram



- Best Product Recommendation



- Product Table with Filters

🔥 Top Discounted Products

Show 10 entries

Search:

product_name	brand_name	marked_price	discounted_price	discount_percent	rating
All	All	All	All	All	All
Bodycon Dress	PURVAJA	3799	569	85	4.4
Bodycon Dress	PURVAJA	3799	569	85	4.4
Black Solid Bodycon Dress	PURVAJA	3799	569	85	4.3
Bodycon Midi Dress	PURVAJA	3799	569	85	4.4
Woman Brown Midi Dress	PURVAJA	3799	569	85	4.2
Woman Brown Midi Dress	PURVAJA	3799	569	85	4.2
Floral Crepe A-Line Dress	Stylo Bug	2663	398	85	4.3
Girls Fit & Flare Dress	Aarika	3499	594	83	4.6
Girls Fit & Flare Dress	Aarika	3499	594	83	4.6
Girls Fit & Flare Dress	Aarika	3499	594	83	4.6

Showing 1 to 10 of 5,659 entries

Previous12345...566Next

- Results / Observations

Overall Dataset Findings

- Total products in dataset: 168,034 across 3,194 unique brands
- Average discounted price across all products: ₹1,515
- Average discount offered: 37.3% — indicating highly competitive pricing
- Average customer rating: 4.17 out of 5.0 — generally well-rated products
- Top product category: T-Shirts with 16,978 listings
- Top brand by listing count: Roadster with 4,982 products

Filter Behaviour Observations

- When Category is changed, the Brand dropdown immediately updates to show only relevant brands, eliminating irrelevant choices
- The Price Range slider automatically rescales to the selected category's min and max prices, making filtering more intuitive and precise
- When all filters are combined (Category + Brand + Price + Rating), results update instantly with no page reload
- The Fallback system ensures the dashboard always displays something meaningful even when filter combinations yield zero products

Chart Observations

- The Top Categories chart shows clothing dominates: T-Shirts, Shirts, Dresses, and Tops account for over 30% of all listings

- The Price vs Discount scatter plot reveals that higher-priced items tend to have higher absolute discounts but not always higher percentage discounts
- The Rating Distribution histogram shows most products cluster in the 3.8–4.5 rating range, with very few below 3.0
- Top Brands chart confirms Roadster, Mast & Harbour, DressBerry, H&M, and max lead in product count

9. Conclusion

This project shows how R Shiny can power an interactive dashboard for practical business analysis. Built around real retail data, the Myntra Smart Dashboard handles 168,034 products with responsive filtering tools. As selections change, performance metrics refresh immediately alongside visual summaries. Insights emerge dynamically, guided by user choices rather than static reports. Product suggestions update in step with applied criteria. Interaction drives discovery here - without reliance on prebuilt narratives.

One focus stands: reactive programming, alongside data cleaning, appears here. A single R app holds these parts together. Data visualization follows next, then user interface layout - each piece shaped by function, not form. During updates, nine issues were resolved over two releases. These corrections reveal how edge cases demand attention. Testing supports stability when systems face unpredictable inputs. Strong validation becomes visible only once flaws emerge. Progress arrives quietly through repeated refinement.

Functioning without limits, this interface appears sleek under low-light conditions due to its shaded layout. Where performance meets readiness, deployment fits immediate operational needs straight out of development. Starting from here, expansion into deeper analytical work becomes possible when handling vast information flows, predictive systems, or live feeds across time. The structure supports next-stage growth naturally, adapting as complexity increases nearby. Built solid, it holds up even when demands shift beyond initial scope.

10. Future

The following enhancements can be implemented in future versions of this project:

- Connect to Myntra's live API to display real-time product data instead of a static CSV file
- Add a machine learning recommendation engine that suggests similar products based on user browsing history
- Implement user authentication so different users can save and compare their own filter preferences
- Add a price trend chart showing how product prices change over time using historical data
- Include a sentiment analysis panel that processes customer review text to compute sentiment scores
- Add more chart types: treemap for category hierarchy, heatmap for brand-category overlap, and box plots for price distribution
- Enable multi-category selection so users can compare two or more categories side by side
- Add a product image viewer panel that loads the product image from `img_link` for the selected best product

- Deploy the dashboard online using shinyapps.io so it is accessible without RStudio
- Add a database backend (SQLite or PostgreSQL) to support faster queries on even larger datasets